WHAM-PFMG474-V4

Firmware Build, Flash, and Verification
Standard Operating Procedure

August 2026

Rev	Date	Author	Description
A	2026-08-31	–	Initial issue: build, bootstrap flash, serial reflash. No power-stage procedures yet (Section 2).

1 Purpose

This document specifies the procedure for building, flashing, and verifying WHAM-PFMG474-V4 firmware, and for confirming a board is running the expected firmware version before it is put into service or handed off.

2 Scope

Covers firmware build and flashing only. **This firmware currently implements no PWM, HRTIM, or power-stage control** – it identifies itself over serial and accepts remote firmware updates, nothing more. This SOP will need a companion procedure for power-stage bring-up, calibration, and safety interlocks once that functionality exists; do not treat this document as covering any high-voltage or power-stage operation.

3 Prerequisites

3.1 Equipment

Item	Notes
WHAM-PFMG474-V4 board	Powered per its own hardware documentation (not covered here).
ST-Link (or equivalent SWD debug probe)	Required for the bootstrap flash (Section 5) and for recovery if a board stops responding over serial entirely.
USB-to-serial adapter	Wired to USART2: adapter TX → PA3 (USART2_RX), adapter RX → PA2 (USART2_TX), GND ↔ GND.

(TX/RX wiring above is crossed: adapter TX goes to the MCU's RX pin and vice versa, as usual for a point-to-point serial link.)

3.2 Software

Tool	Notes
STM32CubeIDE	Project build and the bootstrap SWD flash.
`stm32flash`	AN3155 USART bootloader client. macOS: `brew install stm32flash`.
Python 3 + `pyserial`	Runs `python/wham_serial_flash.py`. `pip install pyserial`.

CAUTION: This firmware has not yet been through an ESD-controlled production process. Use standard ESD precautions (grounding strap, static-safe surface) when handling the bare board.

4 Procedure: Build the Firmware

1. Open the project in STM32CubeIDE and press **Refresh** (`F5`) on the project if any files were added outside the IDE since it was last opened.
2. Build the **Debug** configuration (`Project` → `Build Project`, or from the command line: `cd Debug && make all -j4`).
3. Confirm the build produced no warnings. A clean build produces `WHAM-PFMG474-V4.elf` in `Debug/`.
4. **Generate the `.bin`.** This project's build configuration does not have the "Convert to binary file" post-build step enabled, so it must be generated by hand:

```
arm-none-eabi-objcopy -O binary \
  Debug/WHAM-PFMG474-V4.elf \
  Debug/WHAM-PFMG474-V4.bin
```

5 Procedure: Bootstrap Flash (SWD)

Required the *first* time a board receives firmware with the serial-bootloader feature (`BOOT` command), since that feature is what the serial reflash path in Section 6 depends on – a board cannot use it to receive the very first firmware that adds it. Also use this procedure to recover a board that no longer responds to `BOOT` over serial at all.

1. Connect the ST-Link to the board's SWD header and to the host computer.
2. In STM32CubeIDE, press **Run** (or **Debug**) to build and flash via the ST-Link, as normal for this IDE.
3. Proceed to Section 7 to confirm the board is running the expected firmware.

6 Procedure: Serial Reflash

Use this for all firmware updates *after* the bootstrap flash in Section 5 has been done once. This is the remote-friendly path: no physical BOOT0/NRST/SWD access is required, only the USART2 serial link.

1. With the board powered and running its current firmware, and the USB-serial adapter connected, run:

   ```
   python3 python/wham_serial_flash.py \
     --port /dev/cu.usbserial-XXXXX
   ```

   substituting the actual serial device path. (On macOS, a single USB-serial adapter shows up as both a `cu.*` and `tty.*` device node; the script sees this as two candidates and will not auto-detect – `--port` must be passed explicitly. Prefer the `cu.*` node.)
2. The script will: send `BOOT` and wait for the acknowledgement; run `stm32flash` to write and verify `Debug/WHAM-PFMG474-V4.bin`; then start the new firmware automatically (no manual reset needed – see Section 6.1).
3. Confirm the script reports `[done]` with firmware written and verified before proceeding.
4. Proceed to Section 7.

6.1 Note on automatic restart after flashing

Early testing of this procedure found that the newly-flashed firmware could come up unresponsive over serial until the board was manually power-cycled, because the STM32 ROM bootloader's *Go* command (used to start the new firmware without a physical reset) does not perform a true chip reset, and left the processor's vector table pointer aimed at the bootloader's own vector table rather than the application's. This has been fixed in firmware (`boot_jump.c`) – a manual reset should not be necessary. If a board flashed under this SOP does not respond to the verification step below without a manual reset, that is a regression and should be reported, not treated as expected behavior.

7 Procedure: Verification

1. Open a serial connection to the board at **9600 8N1**.
2. Send `*IDN?` followed by a line ending (`\r`, `\n`, or both).
3. Confirm a reply of the form:

   ```
   OK <board name> <hardware rev> <firmware version>
   ```

   e.g. `OK WHAM-PFMG474-V4 REVA v0.6`.
4. Confirm the reported firmware version matches the version that was just built/flashed.

Acceptance criterion for this SOP: Step 3 above returns a well-formed reply with the expected version, on the first query, with no manual reset performed since the flash in Section 4 or 6.

8 Troubleshooting

Symptom	Cause / action
`stm32flash` reports `Failed to init device, timeout.` before any write occurs	Not a firmware issue – this probe doesn't depend on what's running on the chip. Check board power, serial wiring (TX/RX not crossed, common GND), and that no other program has the port open.
`BOOT` gets no reply, but power/wiring check out	The firmware currently on the board may predate the `BOOT` command (run the bootstrap flash, Section 5) or may be a board affected by the `Go`-jump issue in Section 6.1 – power-cycle once, then retry.
Verification (Section 7) fails after a serial reflash	Power-cycle the board once and repeat verification. If it then succeeds, this is the regression described in Section 6.1 – report it. If it still fails, treat as a failed flash and repeat Section 6 from the start.

9 References

- `AGENTS.md` – project orientation and current functionality.
- `docs/command_reference.md` – full serial command protocol.
- `docs/serial_reflash_guide.md` – detailed reflash mechanism, including the `Go`-jump bug referenced in Section 6.1.
- `docs/changelog.txt` – dated development history.